

Chapitre 2

Analyse Lexicale

Théorie des Langages & Compilation

Dr Tegawendé F. BISSYANDÉ

Master Informatique – UFR/SEA
Université Joseph Ki-Zerbo



Plan du chapitre

- 1 Introduction à l'analyse lexicale
- 2 Théorie des langages formels
- 3 Expressions régulières
- 4 Automates finis
- 5 Transformation NFA $\rightarrow$ DFA
- 6 Minimisation d'un DFA
- 7 Construction de Thompson
- 8 Outils d'analyse lexicale
- 9 Compléments
- 10 Exercices

Objectifs

- **Lire** les caractères du programme source
- **Produire** un flot de mots (*lexèmes / tokens*)

Crible : la suite de lexèmes est ensuite filtrée pour produire une représentation interne : les **unités lexicales**.

Relation avec l'analyse syntaxique :

- Les unités lexicales sont les **données d'entrée** de l'analyseur syntaxique.
- L'analyseur syntaxique appelle l'analyseur lexical à chaque fois qu'il a besoin du prochain token.

Alphabet

Un **alphabet** Σ est un ensemble fini et non vide de caractères.

Exemple : $\Sigma = \{a, b\}$

Mot

Un **mot** (aussi appelé chaîne), ω sur Σ , est une suite finie de caractères de Σ juxtaposés.

Exemple : $\omega = abaab$

- Le mot vide, ne contenant aucun caractère, est noté ε
- $|\omega|$ dénote la longueur du mot ω
- $|\omega|_{\alpha}$ dénote le nombre d'occurrences de la lettre α dans ω

- Ensemble des mots sur Σ de longueur nulle : $\Sigma^0 = \{\varepsilon\}$
- Ensemble des mots sur Σ de longueur n : Σ^n
- Ensemble de **tous** les mots sur Σ :

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$$

- Ensemble des mots non vides sur Σ :

$$\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n = \Sigma^* \setminus \{\varepsilon\}$$

Concaténation et fermeture de Kleene

Concaténation

Si x et y sont deux mots sur Σ , on appelle **concaténation** de x et y le mot $x.y$ (ou xy).

Propriétés : $\alpha.\varepsilon = \varepsilon.\alpha = \alpha$; a^n dénote la concaténation de n occurrences de a .

Fermeture de Kleene

Soit Σ un alphabet, on appelle **fermeture de Kleene** de Σ , noté Σ^* , l'ensemble défini inductivement :

- **Base** : tous les caractères de Σ ainsi que le mot vide ε sont dans Σ^*
- **Règle** : si x et y sont dans Σ^* , alors $x.y$ est dans Σ^*

Définition

On appelle **langage formel** (souvent noté L) sur un alphabet Σ un sous-ensemble de Σ^* .

Opérations sur les langages :

- **Union** : $L_1 \cup L_2 = \{\omega \mid \omega \in L_1 \text{ ou } \omega \in L_2\}$
- **Intersection** : $L_1 \cap L_2 = \{\omega \mid \omega \in L_1 \text{ et } \omega \in L_2\}$
- **Produit** : $L_1 L_2 = \{\omega_1 \omega_2 \mid \omega_1 \in L_1 \text{ et } \omega_2 \in L_2\}$
- **Fermeture de Kleene** : $L^* = \bigcup_{n \geq 0} L^n$
- **Complément** : $\bar{L} = \Sigma^* \setminus L$

Définition inductive

- **Base** : $\emptyset$, $\{\varepsilon\}$ et $\{a\}$ ($\forall a \in \Sigma$) sont des langages réguliers sur Σ .
- **Règle** : Si R_1 et R_2 sont des langages réguliers sur Σ , il en est de même pour :
 - $R_1 \cup R_2$ (union)
 - $R_1 R_2$ (concaténation)
 - R_1^* (fermeture de Kleene)

Il n'existe pas d'autre langage régulier sur Σ .

Définition inductive

- **Base** : $\emptyset$, ε et $a \in \Sigma$ sont des expressions régulières sur Σ , représentant respectivement les langages $\emptyset$, $\{\varepsilon\}$ et $\{a\}$.
- **Règle** : Si r_1 et r_2 sont des expressions régulières décrivant les langages R_1 et R_2 , alors :
 - $(r_1|r_2)$ décrit $R_1 \cup R_2$
 - (r_1r_2) décrit R_1R_2
 - $(r_1)^*$ décrit R_1^*

Exemples d'expressions régulières

Expression rég.	Langage décrit	Éléments
$a b$	$\{a, b\}$	a, b
ab^*a	$\{a\}\{b\}^*\{a\}$	$aa, aba, abba, abbba, \dots$
$(ab)^*$	$\{ab\}^*$	$\varepsilon, ab, abab, \dots$
$abba$	$\{abba\}$	$abba$

Remarque : $a^n b^n$ **n'est pas** une expression régulière. Ce langage n'est pas régulier (il nécessite une grammaire non contextuelle).

Axiome	Description
$r s = s r$	$ $ est commutatif
$r (s t) = (r s) t$	$ $ est associatif
$(rs)t = r(st)$	la concaténation est associative
$r(s t) = rs rt$	2*distributivité par rapport à $ $
$(r s)t = rs rt$	
$\varepsilon r = r\varepsilon = r$	ε est l'élément neutre
$r^* = (r \varepsilon)^*$	relation entre $*$ et ε
$r^{**} = r^*$	$*$ est idempotent

- **Au moins une instance** : $r^+ = r \cdot r^*$
- **Zéro ou une instance** : $r? = r|\varepsilon$
- **Classe de caractères** : $[abc] = a|b|c$; $[a-z] = a|b|\dots|z$

Exemple : les identificateurs du langage C sont engendrés par :

chiffre $\rightarrow [0-9]$

lettre $\rightarrow [a-zA-Z]$

ident $\rightarrow (lettre | _)(lettre | chiffre | _)^*$

Définition

Un automate d'états finis est un quintuplet $A = (Q, \Sigma, \delta, q_0, F)$:

- Q : ensemble fini d'états
- Σ : alphabet d'entrée
- δ : ensemble fini de transitions (arcs étiquetés)
- $q_0 \in Q$: état initial (unique)
- $F \subseteq Q$: ensemble des états finaux (acceptants)

L'automate est représenté par un **graphe orienté** (fini) dont les arcs sont étiquetés par des lettres de Σ ou ε .

Définition

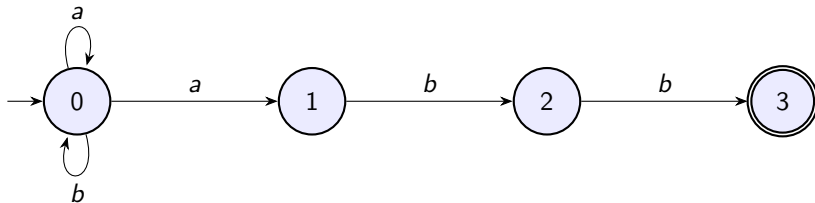
Un automate d'états finis $A = (Q, \Sigma, \delta, q_0, F)$ **accepte** un mot $\omega \in \Sigma^*$ s'il existe (au moins) un chemin dans A allant de q_0 à un état final, étiqueté par les lettres successives de ω (avec possibilité d'intercaler des ε -transitions).

Le langage $L(A)$ reconnu par A est l'ensemble des mots que A accepte.

Caractéristiques du NFA :

- Peut avoir des ε -transitions
- Pour un état et une lettre, peut avoir **plusieurs** états successeurs
- Plus facile à construire, mais plus difficile à simuler

Exemple de NFA



Langage reconnu : $L(A) = (a|b)^*abb$

Table de transitions :

	<i>a</i>	<i>b</i>
0	{0, 1}	{0}
1	∅	{2}
2	∅	{3}

Définition

Un automate fini déterministe (DFA) est un cas particulier de NFA dans lequel :

- ❶ Aucun état n'a de ε -transition
- ❷ Pour chaque état e et chaque lettre a , il y a **au plus un** arc étiqueté a qui quitte e

Avantages : simulation efficace en $O(n)$ où n est la longueur du mot.

Inconvénient : peut avoir un nombre exponentiel d'états par rapport au NFA équivalent.

Entrée / Sortie

- **Entrée** : un NFA $N = (Q, \Sigma, \delta, q_0, F)$
- **Sortie** : un DFA $D = (Q', \Sigma, \delta', q'_0, F')$ tel que $L(D) = L(N)$

Méthode : chaque état du DFA est un **ensemble d'états** du NFA.

Opérations clés :

- ε -**fermeture**(e) : ensemble des états accessibles depuis e uniquement par des ε -transitions
- ε -**fermeture**(E) : union des ε -fermetures des états de E
- **Transiter**(E, x) : ensemble des états accessibles depuis un état de E par une transition sur x

Calcul de $D = (Q', \Sigma, \delta', q'_0, F')$:

- 1 $q'_0 = \varepsilon\text{-fermeture}(q_0)$
- 2 Pour chaque $x \in \Sigma$, ajouter $\varepsilon\text{-fermeture}(\text{Transiter}(q'_0, x))$ dans Q'
- 3 Répéter pour chaque nouvel état ajouté dans Q' jusqu'à stabilité
- 4 $F' = \{E \in Q' \mid \exists e \in E, e \in F\}$

Théorème : Pour tout NFA, il existe un DFA équivalent reconnaissant le même langage. Dans le pire cas, le DFA peut avoir $2^{|Q|}$ états.

Minimisation du nombre d'états

Initialisation

- 1 On retire les états non atteignables
- 2 On partitionne Q en deux classes : états finaux et états non finaux

Principe d'éclatement de partition

- 1 Prendre une classe C_i
- 2 Lui appliquer une transition $x \in \Sigma$
- 3 Séparer les éléments de C_i qui n'aboutissent pas à une même classe
- 4 Continuer jusqu'à obtention d'un **point fixe**

Le DFA résultant est l'automate minimal : il a le **nombre minimum d'états** parmi tous les DFA reconnaissant le même langage.

Expression régulière $\rightarrow$ NFA (construction de Thompson)

Principe : construction inductive à partir de la structure de l'expression régulière.

- $r = \varepsilon$: un seul état initial = final
- $r = a \in \Sigma$: deux états reliés par une transition sur a
- $r = r_1 | r_2$: nouvel état initial avec ε -transitions vers les automates de r_1 et r_2 , puis ε -transitions vers un nouvel état final
- $r = r_1 r_2$: concaténation des deux automates via ε -transitions
- $r = r_1^*$: ajout de ε -transitions pour la boucle et le court-circuit

Pipeline complet de l'analyse lexicale :

Expr. rég. $\xrightarrow{\text{Thompson}}$ NFA $\xrightarrow{\text{Détermination}}$ DFA $\xrightarrow{\text{Minimisation}}$ DFA minimal

Un véritable analyseur lexical est **très gros, très complexe** et **très optimisé**.

Il existe des outils qui, à partir de spécifications basées sur les expressions régulières, génèrent **automatiquement** l'analyseur lexical :

- `lex`, `flex` en C
- `ocamllex` en OCaml
- JavaCC en Java
- ANTLR (multi-langages)

Structure d'un fichier source (OCamllex) :

- **Entête** : code OCaml (commentaires, définitions)
- **Corps** : expressions régulières associées aux actions sémantiques
- **Épilogue** : programme principal

Exemple d'analyseur lexical (OCamllex)

```
(* File lexer.mll *)
{
  open Tokens
  exception Eof
}
rule token = parse
  [ ' ' '\t' ] { token lexbuf }    (* skip blanks *)
| '\n'        { EOL }
| [ '0'-'9' ]+ { INT(int_of_string (Lexing.lexeme lexbuf)) }
| '+'         { PLUS }
| '-'         { MINUS }
| '*'         { TIMES }
| '/'         { DIV }
| '('         { LPAREN }
| ')'         { RPAREN }
| eof        { raise Eof }
```

Problème

Il peut exister des ambiguïtés lors de l'analyse lexicale :

- "let" pourrait être reconnu comme un identificateur
- "lettre" pourrait être reconnu comme LET; IDENT("tre")

Solution : règles de priorité

- 1 Entre deux lexèmes de taille différente : **choisir le plus long** (longest match)
- 2 À longueur égale : suivre l'**ordre de définition** des règles (première règle en premier)

Objectif : supprimer les commentaires dès l'analyse lexicale.

Différents styles de commentaires :

- `// ...` commentaires à la C++ (jusqu'à fin de ligne)
- `/* ...*/` commentaires à la C (non imbriqués)
- `(* ...*)` commentaires à la Caml (imbriqués)

Commentaires à la C++ : ajout d'une règle simple.

Commentaires à la C : utilisation d'un **second automate** `incomment`.

Commentaires à la Caml : ajout d'un **compteur de profondeur** car le niveau d'imbrication est arbitraire.

Le lemme de pompage (pumping lemma)

Énoncé

Soit L un langage régulier. Il existe un entier $n \geq 1$ (longueur de pompage) tel que tout mot $w \in L$ avec $|w| \geq n$ peut s'écrire $w = xyz$ avec :

- ❶ $|y| \geq 1$ (la partie pompée est non vide)
- ❷ $|xy| \leq n$ (la partie pompée est dans les n premiers caractères)
- ❸ $\forall k \geq 0, xy^kz \in L$ (on peut "pomper" y)

Usage : démontrer qu'un langage **n'est pas régulier** par contradiction.

Exemple : $L = \{a^n b^n \mid n \geq 0\}$ n'est pas régulier. Si on prend $w = a^n b^n$, alors $y = a^j$ ($j \geq 1$), et $xy^2z = a^{n+j} b^n \notin L$ car $n+j \neq n$.

Type	Grammaire	Automate	Exemple
3	Régulière	Automate fini	a^*b^*
2	Non contextuelle	Automate à pile	$a^n b^n$
1	Contextuelle	Machine bornée	$a^n b^n c^n$
0	Sans restriction	Machine de Turing	Problème de l'arrêt

Les langages de type k sont strictement inclus dans ceux de type $k - 1$:

$$\text{Réguliers} \subsetneq \text{Non contextuels} \subsetneq \text{Contextuels} \subsetneq \text{R.E.}$$

En compilation : l'analyse **lexicale** utilise le type 3, l'analyse **syntaxique** utilise le type 2.

Expressions régulières en pratique : POSIX et outils Unix

Les expressions régulières sont omniprésentes en informatique :

- **grep** : recherche de motifs dans des fichiers texte
- **sed** : transformation de texte par substitution
- **awk** : traitement de données structurées
- Langages : **Perl**, **Python** (re), **Java** (Pattern), **JavaScript** (RegExp)

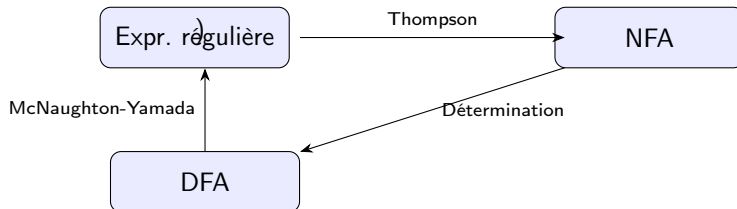
Extensions courantes (au-delà des 3 opérations de base) :

Notation	Signification
$[^s]$	Tout caractère sauf ceux de s
$r\{n\}$	Exactement n répétitions de r
$r\{n,m\}$	Entre n et m répétitions de r
$\backslash d, \backslash w, \backslash s$	Chiffre, alphanumérique, espace
$.$	N'importe quel caractère

Théorème de Kleene

Pour un langage L , les trois propositions suivantes sont équivalentes :

- 1 L est reconnu par un **automate fini déterministe** (DFA)
- 2 L est reconnu par un **automate fini non déterministe** (NFA)
- 3 L est décrit par une **expression régulière**



Exercice 1 – Langages et expressions régulières

- ❶ Soit $\Sigma = \{a, b\}$. Décrire les langages dénotés par :
 - $a(a|b)^*$
 - $(a|b)^*a(a|b)^*a(a|b)^*$
 - $a^*ba^*ba^*$
- ❷ Donner une expression régulière pour chacun des langages suivants sur $\Sigma = \{0, 1\}$:
 - L'ensemble des chaînes ne contenant pas deux 1 consécutifs
 - L'ensemble des chaînes dont le troisième caractère en partant de la fin est 1
- ❸ Montrer que $a^n b^n$ ($n \geq 0$) n'est pas un langage régulier (indication : utiliser le lemme de pompage).

Exercice 2 – Automates finis

- 1 Construire un NFA pour l'expression régulière $(a|b)^*abb$ en utilisant la construction de Thompson.
- 2 À partir du NFA obtenu, appliquer l'algorithme de détermination pour construire le DFA équivalent. Donner la table de transitions et dessiner le DFA.
- 3 Appliquer l'algorithme de minimisation au DFA obtenu. Donner le DFA minimal résultant.

Exercice 3 – Analyse lexicale pratique

- ❶ Écrire les expressions régulières pour les unités lexicales suivantes du langage C :
 - Identificateurs
 - Entiers (décimaux et hexadécimaux)
 - Nombres flottants (avec partie décimale et/ou exposant)
 - Chaînes de caractères (entre guillemets, avec échappement \)
- ❷ Donner la suite de tokens produite par l'analyse lexicale du code C suivant :

```
if (x >= 10) { y = x * 2 + 3.14; }
```

Exercice 4 – NFA et DFA

Soit le NFA suivant avec $\Sigma = \{a, b\}$, $q_0 = 0$, $F = \{2\}$:

État	a	b	ε
0	$\{1\}$	$\emptyset$	$\{3\}$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\{2\}$	$\{2\}$	$\emptyset$
3	$\emptyset$	$\{4\}$	$\emptyset$
4	$\{2\}$	$\emptyset$	$\emptyset$

- 1 Décrire en français le langage reconnu par cet automate.
- 2 Construire le DFA équivalent par l'algorithme de détermination.
- 3 Donner une expression régulière équivalente.